

Laboratorio 10: Vectorización, Indexación y Reconocimiento de Rostros

Introducción

Las imágenes, y específicamente los rostros, representan uno de los tipos más complejos de datos no estructurados, requiriendo técnicas especializadas para su procesamiento y recuperación eficiente.

El tratamiento de datos biométricos faciales presenta varios retos:

- La necesidad de convertir la información visual del rostro en una representación vectorial procesable.
- El almacenamiento eficiente de vectores de alta dimensión.
- La implementación de búsquedas eficientes y precisas en grandes colecciones de datos.

Este laboratorio aborda estos retos explorando diferentes técnicas de indexación vectorial para optimizar la búsqueda de rostros similares.

El proceso general consiste en:

1. **Vectorización de rostros:** Utilizando la biblioteca `face_recognition`, convertiremos imágenes de rostros en vectores característicos (embeddings) de 128 dimensiones que capturan los rasgos faciales distintivos.
2. **Indexación:** Exploraremos la extensión PyVector, que permite emplear distintas estructuras de indexación en espacios de alta dimensión para almacenar y buscar vectores de manera eficiente. En particular, analizaremos los métodos IVF (Inverted File Index) y HNSW (Hierarchical Navigable Small World), ampliamente utilizados en tareas de búsqueda aproximada de vecinos más cercanos.
3. **Búsqueda de Similitud:** Implementaremos la técnica k-NN para identificar los rostros más cercanos a una imagen de consulta. Compararemos la eficiencia y la calidad de los resultados obtenidos mediante búsqueda indexada frente a los de una búsqueda lineal.

P0. Cargar el Dataset de Rostros

Instalación de dependencias

Primero asegurate de tener instalado Python 3.9 o 3.10, luego instala la librería `face_recognition`:

```
# instalar con pip
pip install dlib
pip install face_recognition

# instalar con conda
conda create -n vision python=3.9
conda activate vision

conda install -c conda-forge dlib
conda install -c conda-forge face_recognition
```

Nota: Para Windows se recomienda usar `conda` para facilitar la instalación de las dependencias.

Descarga del dataset

Descargar [LFW Dataset](#) y descomprimir en una carpeta localmente.

```
import os
import glob
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from rtree import index
import face_recognition

DATASET_PATH = "D:\\temp\\lfw"

coleccion = []
for path in glob.iglob(os.path.join(DATASET_PATH, "**", "*.jpg")):
    person = path.split("\\")[-2]
    coleccion.append({"person": person, "path": path})

coleccion = pd.DataFrame(coleccion)
coleccion.head(10)

def mostrarFotos(coleccion, posiciones):
    plt.figure(figsize=(16,10))
    i = 0
    for idx in posiciones:
        img = plt.imread(coleccion.path.iloc[idx])
        plt.subplot(4, 4, i+1)
        plt.imshow(img)
        plt.title(coleccion.person.iloc[idx]+str(img.shape))
        plt.xticks([])
        plt.yticks([])
        i += 1
    plt.tight_layout()
    plt.show()

posiciones = list(range(0, 16))
mostrarFotos(coleccion, posiciones)
```

P1 (4 pts). Generar los vectores característicos

Implementar la función `generate_face_embeddings()` el cual recibe la colección de imágenes y el número N de rostros a procesar:

1. **Extracción de características:** Usando la biblioteca `face_recognition`, cada imagen se procesa para extraer un vector de 128 dimensiones que captura las características faciales.

```
# Leer imagen
image = face_recognition.load_image_file(filename)
# Detectar rostros en la imagen
face_encodings = face_recognition.face_encodings(image)
if face_encodings:
    # Tomamos el primer rostro detectado en la imagen
    embeddings.append(face_encodings[0])
```

2. **Almacenamiento:**

- o Para evitar reprocesar las imágenes en cada ejecución, guardar los vectores generados en PostgreSQL con la extensión `pgvector`

```
docker run -d --name pgvector -e POSTGRES_PASSWORD=123456 -p 5433:5432 ankane/pgvector
```

- o Usando Python cargar toda la colección de fotos con su respectivo embedding.
- o El tipo de dato `vector` puede almacenar vectores de mas de 100 dimensiones.

```
-- Habilitar la extensión pgvector
```

```
CREATE EXTENSION IF NOT EXISTS vector;
```

```
-- Crear una tabla para almacenar los embeddings faciales
```

```
CREATE TABLE IF NOT EXISTS face_embeddings (
    id SERIAL PRIMARY KEY,
    name TEXT,
    path TEXT,
    embedding VECTOR(128) -- Vectores de 128 dimensiones
);
```

P2 (3 pts). Búsqueda KNN Lineal

Implementar búsqueda kNN secuencial sin índices:

- Cargar la imagen de consulta y extraer el embedding. Se recomienda una foto que no sea parte de la colección.
- Comparar el vector de consulta con cada embedding de la base de datos usando distancia euclidiana $\langle - \rangle$ y coseno $\langle = \rangle$
- Ordenar resultados por distancia y retornar los k más cercanos
- Apuntar los tiempos y analizar los resultados obtenidos ¿Con ambas distancias se obtiene el mismo resultado?

P3 (7 pts). Búsqueda KNN usando PyVector de PostgreSQL

En este paso se le pide implementar la búsqueda KNN sobre los datos indexados. `pgvector` proporciona soporte nativo para búsquedas eficientes en vectores de alta dimensión (IVF Flat, HNSW).

```
-- Ejemplo de creación de índice IVFFlat para distancia Euclidean
CREATE INDEX IF NOT EXISTS face_embedding_index
  ON face_embeddings USING ivfflat (embedding) WITH (lists = 100);

-- Ejemplo de creación de índice IVFFlat para distancia Coseno
CREATE INDEX IF NOT EXISTS face_embedding_index
  ON face_embeddings USING ivfflat (embedding vector_cosine_ops) WITH (lists = 100);
```

Búsqueda por similitud:

- Utilizar el operador <-> o <=> para calcular la distancia.
- Ejecutar consultas KNN aprovechando el índice vectorial
- Recuperar los rostros más similares ordenados por distancia
- Apuntar los tiempos y analizar resultados respecto a la búsqueda lineal

```
-- Ejemplo de KNN indexado
SET ivfflat.probes = 10; -- centroides (clusters) visitados durante una búsqueda

SELECT id, name, embedding <-> '[0.12, 0.34, ..., 0.99]' AS distance
FROM face_embeddings
ORDER BY embedding <-> '[0.12, 0.34, ..., 0.99]'
LIMIT 5; -- Top 5
```

P4 (6 pts). Análisis de resultados

1. Efectividad

- Comparar las imágenes que retorna cada búsqueda indexada (KNN con IVF, KNN con HNSW) con los resultados obtenidos por el K-NN lineal. ¿Cual de los dos métodos de indexación se acerca más al resultado de la búsqueda lineal?
- Interpretar por qué PgVector puede diferir de la búsqueda lineal

1. Rendimiento

- Usar EXPLAIN ANALYZE para medir tiempos de ejecución
- Probar con diferentes tamaños del dataset: 1000, 2000, 4000, 8000, 12000 embeddings
- En un solo gráfico comparar tiempo de consulta para las dos búsquedas lineal e indexada (KNN Lineal, KNN con IVF, KNN con HNSW)

3. Conclusiones

- Analizar ventajas y desventajas de cada método
- Evaluar escalabilidad de dichas técnicas con datasets muy grandes
- Identificar casos reales diferentes a Face Recognition que se beneficien de una base de datos vectorial optimizada

Entregable:

- Informe con evidencias de cada apartado

